

Typed Decisions in Agent Memory: Where They Help, Where They Don't, and What It Costs

[Author(s) TBD]

Code: [REPO URL PLACEHOLDER]

Abstract

Typed decisions cut an agent memory system's decision cost and latency and help retrieval under a small budget; they do not change answers after facts change, and a small zero-shot model cannot make them. Memory systems make an LLM call for every write decision, which makes revisiting the store unaffordable, yet most of that work is choice among fixed options. In engram, an LLM extracts facts and every decision is a typed question answered by a hosted decision model (Jev). With extraction held identical to mem0 2.1.0's, typed decisions cut decision-layer cost by 70.0× and median decision latency by 27.6×, with equal dev accuracy (31/35 against 31/35). On 610 held-out LoCoMo questions under a three-memory retrieval budget, engram answers +8.7 points more than mem0 at matched context tokens (95% CI +5.2 to +12.1); at k=20 the systems tie (+0.8, CI -2.3 to +3.9). A belief-state store policy over-closed no labeled keep item. Two results are negative: closing stale facts does not change answers on current benchmarks, and the 421M base checkpoint of the open-weights decision model Laya, used zero-shot as its documentation advises against, does not make the relational decisions. Code, prompts and per-question results are released.

1 Introduction

The cost problem. A memory system for an LLM agent turns conversation into facts it can retrieve later. Each write has two parts:

- Extraction: what facts does this message state?
- Decisions: is each fact new, a duplicate, a refinement, or a change to something already stored? Is it worth keeping? Is it sensitive?

In current open-source systems both parts are LLM calls. mem0 2.1.0's default add() makes a single LLM call per message with its additive extraction prompt, and has no separate update or delete step (mem0/memory/main.py, Memory._add_to_vector_store; prompts reproduced in Appendix B). An LLM call per decision is expensive enough that nothing re-examines the store afterwards. Facts that stopped being true stay in it.

The observation. The decisions are choices among

options fixed in advance. That is the setting typed decision models are built for. They return a probability over a fixed option set in one short request, and they cost far less than generating text.

Concurrent work. The architectural idea of routing memory decisions to a typed decision model was reached independently by Jev-Mem [8], which uses Jev for typing, relation construction, query routing, budget allocation, traversal, candidate scoring and stopping. It reports a LoCoMo judge score of 0.777 with a 158 s build and 0.93 s query against A-MEM [21], Nemori, MemoryOS and MAGMA [9]. It does not isolate the decision layer, evaluate updates or closes, measure calibration, or use a held-out split or confidence intervals. Separately, a community article reranked AtMem's top-10 with Jev on 1,986 LoCoMo questions and measured the effect at the ranking level: MRR@5 rose from 0.4259 to 0.5868 and Recall@1 from 0.3399 to 0.5423 [18]. This paper's contribution is the controlled measurement: an identical-extraction ablation of the decision layer, its calibration, the safety of the store it drives, the rerank effect on answers with a matched-context control, and the negatives.

Contributions.

1. With extraction held identical, typed decisions replace an LLM decision layer at 70.0× lower cost and 27.6× lower median latency with no measured accuracy loss (§5.1).
2. A belief-state store policy with reversible, gated closes over-closes no labeled keep item, and its v3 rule removes a weak-evidence failure that closes true facts (§3.4, §5.3).
3. Under a three-memory budget a listwise Jev rerank puts engram +8.7 points ahead of mem0 at matched context on 610 held-out questions, and the systems tie at k=20 (§5.2).
4. Closing stale facts does not change answers on current benchmarks, and a zero-shot base checkpoint of an open-weights decision model does not make the relational decisions (§5.5, §5.6).

Scope. We compare against one baseline (mem0 OSS 2.1.0) on one benchmark (LoCoMo: one development conversation and four held-out conversations, scoring four of its five question categories; adversarial is excluded, following mem0's evaluation protocol). We use update

sets written by the system’s author, and one hosted decision model (Jev, jev-1.13.0).

We did not test:

- Zep/Graphiti or Letta
- LongMemEval
- other extraction, answer or judge models
- multi-user stores
- non-English text

2 Background

2.1 How memory systems make write decisions

- **mem0** [4] (2.1.0) extracts facts with one LLM call per message. The prompt receives the new message, the last 10 messages and the 10 most similar existing memories, and emits only additions. Its older update step, which asked an LLM to label each new fact as ADD, UPDATE, DELETE or NONE against existing memories, is still shipped as DEFAULT_UPDATE_MEMORY_PROMPT. We use it as the LLM decision layer in §5.1.
- **Zep’s Graphiti** resolves entities and invalidates edges with LLM calls [16].
- **ByteRover** curates a hierarchical context with an LLM; its retrieval is a five-tier progressive strategy that answers most queries in under 100 ms without LLM calls and escalates to agentic reasoning only for novel questions [13].

2.2 Typed decision models

A typed decision model takes a shared *state* (JSON) and a set of questions. Each question is one of:

- a choice among named options, each with a rubric
- a yes/no (“noul”) question
- a score

It returns a probability distribution per question.

Jev. We use Jev through TypeSafe’s API: model jev-1.13.0, priced at 0.042 USD per million input tokens (src/engram/config.py). Its documentation gives a limit of 255 options per choice question [19]. Measured latency is flat in request size (§5.7).

Laya. Laya is an open-weights model with the same request format. We ran checkpoint convaiinnovations/laya locally through the laya-mlx port on an Apple M2 Pro with 32 GB. It reads at most 512 tokens per question, of which at most 192 go to the instructions and options. This is the 421M base checkpoint. Its model card reports zero-shot typed-decision accuracy of 0.362 against a 0.318 random baseline, calls it “a fast base to specialise, not a zero-shot decision engine,” and offers a separate

checkpoint fine-tuned for typed decisions [5]. We used the base checkpoint zero-shot.

2.3 LoCoMo and its limits

LoCoMo [11] contains long multi-session conversations with questions in five categories, of which we score four (adversarial is excluded, following mem0’s evaluation protocol; §4.1).

It barely tests updates. In the first 215 messages of conv-26, an LLM labeler (claude-sonnet-4-6, prompt in bench/stale.py) found 2 claims that a later message makes no longer true (bench/slices/conv26_superseded.json).

Public scores are not comparable. Published LoCoMo scores for the same systems differ between the systems’ own reports and third-party reports [2, 12]. We do not quote those numbers. All comparisons here run both systems under one protocol.

3 engram

3.1 Pipeline

Write path.

1. An LLM extracts facts from a message. In the experiments this uses mem0’s prompt and inputs; §4.2.
2. Each fact goes to Jev as a single request. The request carries the fact questions and one *relation_to_candidate* question for each of up to 10 candidate facts.
3. A policy layer (§3.3) and a belief state (§3.4) turn the answers into store operations.
4. The store is SQLite plus NetworkX. Facts are edges with validity windows, and every decision is logged with its probabilities.

Read path.

1. Embed the question and take the cosine top-k.
2. Jev reranks the candidates with one yes/no relevance question per candidate, plus a *query_relation* question that can pull in facts by relation type.
3. Add history: the facts each retrieved fact superseded.
4. Render compact lines for the answer model: the date the fact was said, the fact, and the verbatim source quote.

Figure 2 walks through these stages on a question from the retrieval regression test.

3.2 The decision chain

Table 1 lists the 12 questions in the current chain; Appendix A gives the full option text of every version. Choice questions carry a rubric per option. When a question’s wording changes, its version number is bumped and old arms keep the old version.

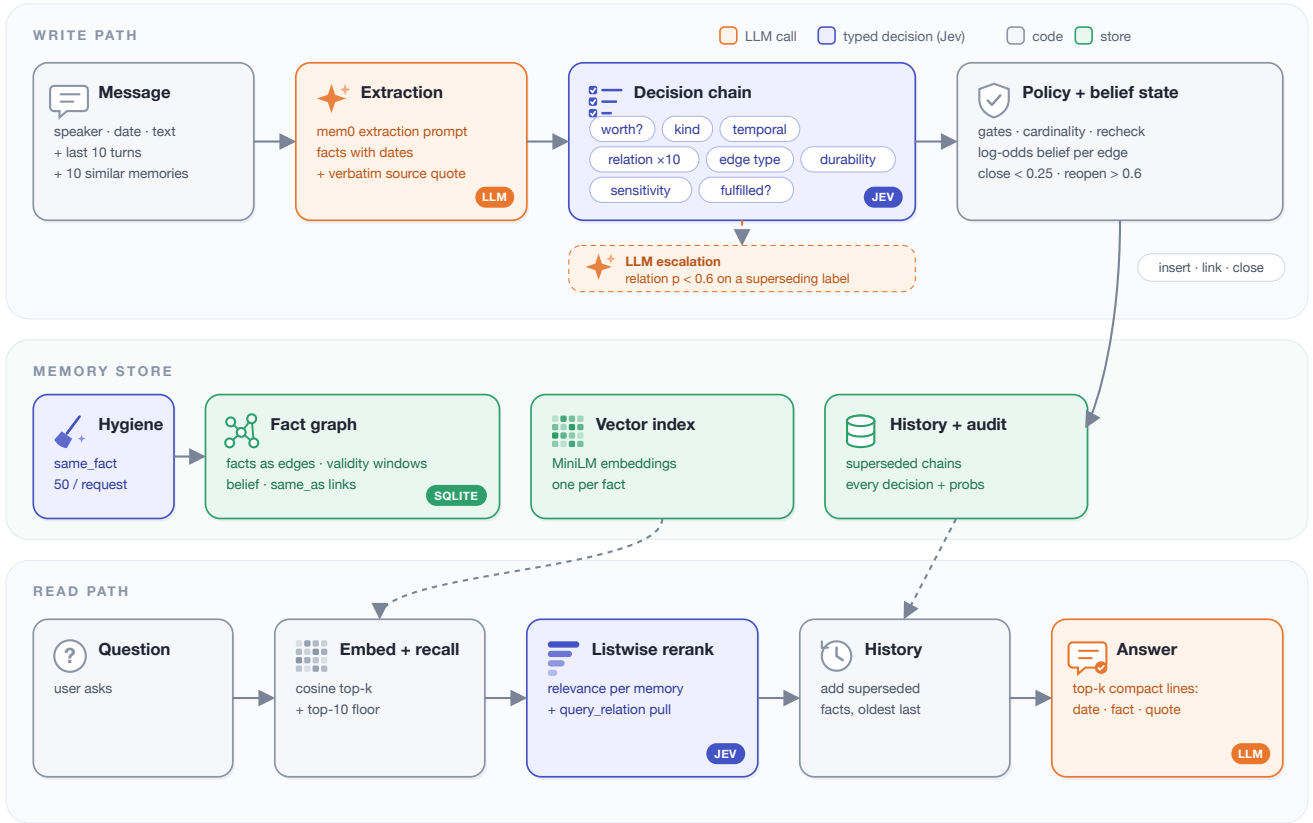


Figure 1: engram’s write and read paths. Orange boxes are LLM calls, blue boxes are typed Jev decisions.

3.3 Policy layer

Decisions act only through explicit rules (src/engram/pipeline/write.py):

- **Thresholds.** A decision acts at $p \geq 0.85$. A superseding relation (update, contradiction, negates) below 0.60 is escalated to an LLM, using mem0’s update prompt. Anything in between is stored as tentative.
- **Temporal gate.** An old edge can close only if the new fact is current or past: $P(\text{current}) + P(\text{past}) \geq 0.85$. Planned and hypothetical statements never close anything.
- **Cardinality.** Each relation type is single- or multi-valued. contradiction closes only a sibling (same subject and relation) on a single-valued relation. update may also close a multi-valued sibling. negates (the new fact says the old one stopped) may close any edge.
- **Two-phrasing agreement.** The first piece of evidence against an edge must be confirmed by a second phrasing of the relation question (relation_to_candidate_recheck). Otherwise it does not count.
- **Plans.** A dedicated yes/no question, plan_fulfilled, asks whether a new fact reports that a stored plan has happened. At $p \geq 0.85$ the plan closes with reason *fulfilled*.

- **Merges are links.** A duplicate becomes a *same_as* link, collapsed at retrieval and removable. Text is never merged.
- **Overrides.** An explicit request to remember overrides *worth_remembering*. A credential is redacted before storage, whatever the other answers say.

3.4 Belief state

Each edge i carries a belief $b_i \in [0.02, 0.98]$ that it is currently true. A typed answer z about the edge, with probability q for its chosen label, updates it in log-odds:

$$\text{logit } b_i \leftarrow \text{logit } b_i + w(z) \text{logit } q, \quad (1)$$

$$\text{logit } q \leftarrow \text{logit } q / T. \quad (2)$$

With a per-question temperature T (§5.4), logit q is first rescaled as in Eq. (2). We use $|w| = 1$. The sign and the zero cases are exactly the policy rules of §3.3:

- $w = +1$ for duplicate and refinement (support)
- $w = -1$ for update, contradiction and negates where §3.3 permits a close
- $w = 0$ for new, and for any answer a gate blocks

An open edge closes when $b_i < 0.25$ and reopens when $b_i > 0.60$. The gap between the two gives hysteresis.

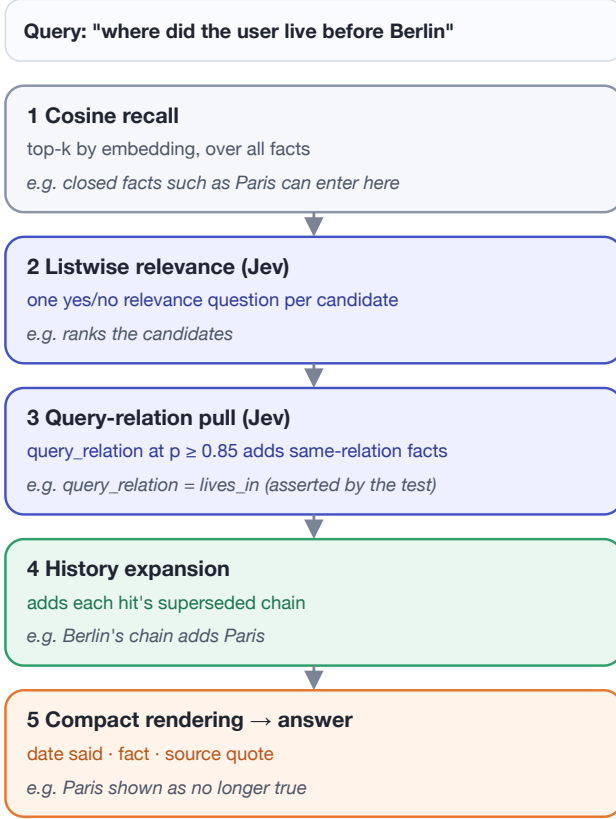


Figure 2: The read path on “where did the user live before Berlin” (fixture in `tests/test_retrieval_regression.py`). Cosine recall searches closed facts too, so Paris can enter at stage 1 or through Berlin’s superseded chain at stage 4; the test asserts that Paris is retrieved and that Acme, another closed chain, is not, but no stored run records which stage added Paris.

Belief v3. The v2 rule used in the held-out runs lets any answer count. A duplicate at $q < 0.5$ therefore has logit $q < 0$ and *lowers* belief, and a weak against-answer raises it. v3 counts an answer only when its label is the argmax and $q > 0.5$ (§5.3). Figure 3 traces one fact from the dev run under both rules.

What this assumes. The log-odds rule Eq. (1) treats q as a calibrated likelihood. Where the model is miscalibrated, belief moves by the wrong amount. This is why §5.4 measures calibration and why a per-question temperature is part of the rule.

3.5 Hygiene pass

After ingestion, one pass re-checks every group of facts that share a subject and relation. It asks the yes/no question `same_fact` for each pair, 50 pairs per request. At $p \geq 0.85$ the pair is linked, and at $p \leq 0.15$ an existing link is dropped.

On the held-out conversations one pass made between 1,865 and 5,446 decisions, for between \$0.0112

Table 1: Jev questions (`src/engram/decide/questions.py`, `docs/DECISIONS.md`). `edge_type` and `query_relation` choose among 25 relation types.

question	type	options
worth_remembering	noul v1	yes / no
fact_kind	choice v1	preference, bio, event, relationship, task, opinion
temporal_status	choice v2	current, planned, past, hypothetical
relation_to_candidate	choice v2	new, duplicate, update, contradiction, refinement, negates
relation_to_candidate_recheck	choice v2	new, duplicate, update, contradiction, refinement, negates
edge_type	choice v1	25 relation types
durability	choice v1	permanent, long_term, short_lived
sensitivity	choice v1	none, health, financial, relationship, credentials
plan_fulfilled	noul v1	yes / no
relevant_to_query	noul v1	yes / no
query_relation	choice v1	25 relation types
same_fact	noul v1	yes / no

and \$0.0332 (§5.3). With LLM decisions, re-examining the store at that scale is what becomes unaffordable.

3.6 Cost model

With a per-decision Jev cost c_J , an LLM escalation cost c_L and an escalation threshold θ on the top probability q :

$$C(\theta) = c_J + P(q < \theta) c_L, \quad (3)$$

$$E(\theta) = P(q \geq \theta) \varepsilon_J(\theta) + P(q < \theta) \varepsilon_L, \quad (4)$$

where $\varepsilon_J(\theta)$ is Jev’s error rate on the decisions it keeps and ε_L is the LLM’s error rate on the escalated ones. §5.4 draws the empirical curves of Eqs. (3) and (4).

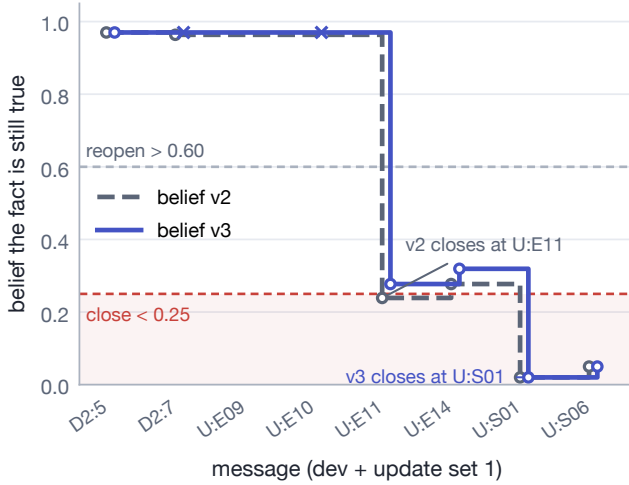


Figure 3: Belief in the fact “Melanie carves out daily me-time through running, reading, or playing violin” (message D2:5), dev + update set 1, under v2 and v3. Under v2 a weak refinement answer at D2:7 ($p = 0.45$) lowers belief slightly, so the update at U:E11 takes it below the close line; v3 ignores that answer (×) and the fact closes one message later, at U:S01. Source: the belief trace in `bench/results/e4_belief_v2__dev_updates__k3__noanswer.json` and its v3 counterpart.

4 Experimental setup

4.1 Models, data, caching and budget

Models (the same in every arm):

- Extraction: `claude-haiku-4-5`.
- Answers and judging: `claude-sonnet-4-6`, with `mem0`’s LoCoMo evaluation prompts (Appendix B).
- The LLM decision layer and all escalations: `claude-sonnet-4-6` with `mem0`’s update prompt.
- `mem0 2.1.0` runs on `claude-haiku-4-5` with telemetry off.

Slices:

- *dev*: conv-26 sessions 1–4, 76 messages, 35 questions.
- *stress*: conv-26 sessions 1–10, 215 messages, 80 questions.
- *held-out*: conv-30, conv-41, conv-42 and conv-43 whole, 610 questions. These were not used during development. The system configuration was frozen (git tag `e4-frozen`) before any held-out run.

LoCoMo’s questions fall in five categories, of which we score four (adversarial is excluded, following `mem0`’s evaluation protocol). Each held-out conversation is ingested once per system, and $k=3$ and $k=20$ are answered from the same store.

Caching and budget. Every LLM and Jev call is cached by its full request, and budget guards stop any run past a spending limit. Phase 2 (all experiments reported here) spent \$93.26 over 80 ledgered runs. Jev accounts for

\$1.47 of it (`bench/results/phase2_spend.jsonl`; per-arm totals in Appendix E). The build phase was not ledgered; roughly \$10 by the author’s estimate.

4.2 The identical-extraction protocol

For each message, `mem0 2.1.0`’s `add()` builds its extraction prompt from:

- the new message, as `[date] speaker: text`
- the last 10 messages
- the 10 existing memories most similar to it
- a current date

It passes no observation date, so relative dates in historical conversations resolve against the current date. We report this as-is, and a patched variant in §6.

engram’s E-arms build the same prompt with the same function (`generate_additive_extraction_prompt`), inputs and model (`bench/run.py`, `src/engram/flags.py`). The “existing memories” differ only because each system’s store differs. The E2 arms differ only in what decides after extraction.

4.3 Update sets

LoCoMo has almost no updates (§2.3). Two update sets, written in the speakers’ voices, extend it. Both are appended to the dev slice and were frozen by SHA-256 before any run.

Set 1 (`bench/updates_conv26.json`, SHA-256 prefix `2330876388f1`) has 30 items:

- 15 easy closes
- 10 subtle items labeled *no_close*: past-tense mentions and unrealised plans
- 5 fulfilled plans

Set 2 (`bench/updates2_conv26.json`, prefix `d4f3c8d2de61`) adds 28 messages and 20 questions:

- 5 point-in-time questions
- 5 chains of 3–5 changes asked about now
- 5 chains asked about a past moment
- 5 updates with no temporal cue in the message

All items are in Appendix C. The system’s author wrote and labeled them, and this is a source of bias.

4.4 Statistics

Both systems answer the same questions, so comparisons are paired: $d_i = \text{engram}_i - \text{mem0}_i$. We report:

- an exact McNemar test on the discordant questions
- a 95% interval from the per-question normal approximation

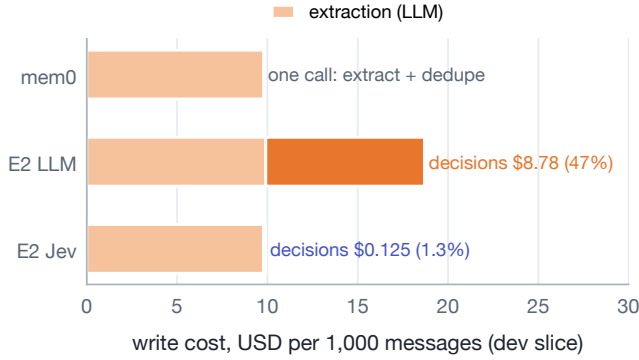


Figure 4: Write cost per thousand messages on the dev slice (76 messages), split into extraction and decision layer. mem0 makes one call per message that extracts and deduplicates.

- a cluster bootstrap that resamples the four held-out conversations. With four clusters, treat it as a robustness check.

The token-matched comparison reports the per-question interval only.

5 Results

5.1 Replacing the decision layer

The three arms in Table 2 share extraction. The two engram arms differ only in what decides after it:

- Jev typed questions (E2 Jev)
- claude-sonnet-4-6 with mem0’s update prompt, one call per extracted fact (E2 LLM)

The two decision layers answer the same number of questions and mem0 one fewer, within noise. The Jev decision layer costs $70.0\times$ less and has $27.6\times$ lower median decision latency. The LLM decision layer stores fewer facts (18 against 47) because mem0’s UPDATE event rewrites an existing memory instead of adding one.

The two latency columns are medians over different messages. In the E2 LLM arm only 30 of 76 messages made an LLM decision call, so the write median falls on an extraction-only message (median extraction 869 ms); on the messages with decisions, the logged median of the slowest decision call is 7,892 ms (bench/e2_latency.py).

Figure 4 shows where each system’s write cost goes: extraction dominates, and the decision layer is 1.3% of the Jev arm’s cost against 47.0% of the LLM arm’s.

These ratios compare Jev against claude-sonnet-4-6 as the decider. We have no measurement with a smaller LLM decider. On the stress slice the Jev arm scored 67/80 and mem0 64/80; the LLM arm was not run there.

Held-out parity at default k. The held-out runs compare the frozen full system (E4 belief v2, §3.4) with mem0, not the E2 arms. At k=20 engram answers 482/610 and mem0 477/610. The difference is +0.8 points

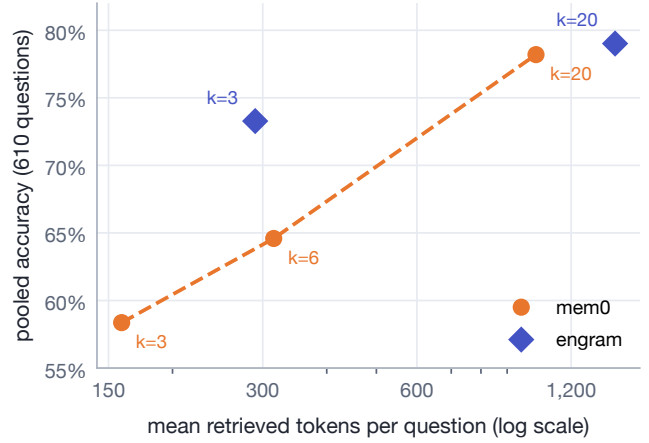


Figure 5: Pooled held-out accuracy (610 questions) against mean retrieved tokens per question. mem0 accuracy was measured at three settings, $k = 3, 6$ and 20; the token-matching sweep counted tokens at the other k without answering. Models as in Table 3.

(95% CI -2.3 to +3.9; conversation bootstrap -4.5 to +5.4; McNemar $p = 0.679$), within noise.

5.2 Retrieval under a small budget

Our answer-level result is consistent with the ranking-level improvement AtMem measured when reranking with Jev [18].

At k=3 engram shows the answer model 290 tokens per question and mem0 159. Most of the difference is the source quote on each engram line. To separate context size from ranking, we answered the same 610 questions from mem0’s existing held-out stores at every k from 3 to 8. The token-matched setting is the k whose mean retrieved tokens came closest to engram’s 290: $k=6$, at 315 tokens. That gives mem0 slightly more context than engram. The run cost \$2.52.

At k=3 engram is ahead of mem0 by +14.9 points (95% CI +11.2 to +18.7). Against token-matched mem0 the difference is +8.7 points (95% CI +5.2 to +12.1; McNemar $p = 1.3e-06$; 86 questions only engram answered against 33 only mem0 answered).

Figure 5 places the three measured mem0 settings and the two engram settings on one axis of retrieved tokens. engram at k=3 sits above the line through mem0’s points, and the two systems meet at k=20.

engram’s point estimate is ahead in every conversation and in every category at matched context (Figure 6, Table 5, Figure 7). Per conversation, the matched-context interval excludes zero in 2 of four conversations (conv-42 and conv-43); conv-30 and conv-41 are within noise on their own.

Attribution. Matching context removes 41.8% of the k=3 difference. The other 58.2% is what remains once context is matched.

We attribute that remainder to which memories reach

Table 2: Dev slice, conv-26 sessions 1–4, 76 messages, 35 questions, all retrieved memories (default k). Extraction claude-haiku-4-5 for all arms; answers and judge claude-sonnet-4-6; E2 LLM decides with claude-sonnet-4-6. Costs are per 1,000 messages written. Decision p50: median time after extraction, per message, over messages that produced at least one fact. Write p50: median end-to-end write time per message over all messages, including those that produced no facts.

system	accuracy	decision \$/1k	decision p50	total \$/1k	write p50	facts
mem0 2.1.0 (one LLM call per write)	30/35	–	–	\$9.80	879 ms	46
engram E2, LLM decision layer (mem0 update prompt)	31/35	\$8.782	7,675 ms	\$18.70	918 ms	18
engram E2, Jev decision layer	31/35	\$0.125	278 ms	\$9.90	895 ms	47

Table 3: Held-out LoCoMo accuracy: conv-30, 41, 42 and 43, 610 questions, adversarial category excluded, k as labeled. Extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6. One ingestion per system per conversation. Tokens are mean retrieved-context tokens per question.

system	k	tokens/q	conv-30	conv-41	conv-42	conv-43	pooled
Q			81	152	199	178	610
engram	3	290	72.8%	74.3%	75.4%	70.2%	447/610
mem0	3	159	67.9%	58.6%	55.8%	56.7%	356/610
mem0 (token-matched)	6	315	66.7%	71.7%	59.8%	62.9%	394/610
engram	20	1,461	76.5%	84.2%	78.4%	76.4%	482/610
mem0	20	1,024	77.8%	88.8%	73.4%	74.7%	477/610

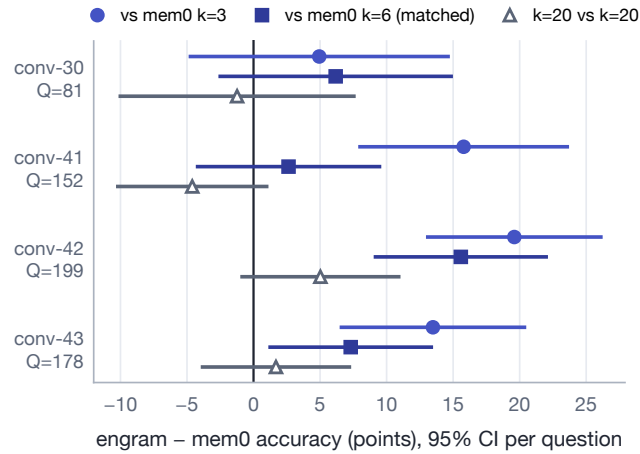


Figure 6: engram minus mem0 accuracy per held-out conversation, with per-question 95% intervals: engram k=3 against mem0 k=3, against token-matched mem0 k=6, and both at k=20. Source: bench/results/perconv_diffs.json.

the top three, which rests on a dev-slice ablation, not a held-out one. On dev + set 1 at k=3 the frozen arm answered 30/30 update questions. It answered 26/30 with Jev reranking off, 30/30 with history expansion off, and mem0 answered 25/30. We did not run a held-out no-rerank arm.

5.3 Safety and cost of the store

Figure 8 shows the same outcomes per arm.

Over-closes. No labeled keep item was over-closed by any arm.

Closes outside the labels. They separate the policies:

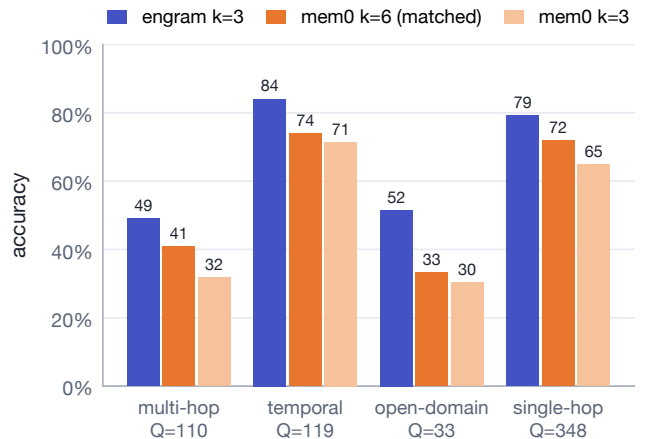


Figure 7: held-out accuracy by category at k=3, for engram, token-matched mem0 (k=6) and mem0 (k=3).

- The first Jev arm (E2), which closed whenever a superseding label cleared 0.85, made 11 closes on dev + set 1. 8 of them match no labeled pair.
- Belief v2 made 4, of which 1 match no labeled pair.
- On set 2 the same comparison is 10 of 14 against 1 of 10.

Stale items. A stale item is a close item whose old fact is still active. Belief v2 left 14/18 of the set-1 close items stale, against 17/18 under belief v1 and 18/18 for mem0, which never closes. On set 2 it left 10/16 stale, against 15/16 and 16/16.

Fulfilled plans. A relaxed heuristic closed a plan whenever a related past or current fact arrived. Across its two arms it fired 25 times. 1 firing matched a labeled (plan, fulfilment) pair, and 24 did not. The frozen arm uses the plan_fulfilled question instead; it was asked

Table 4: Paired differences for the rows of Table 3: held-out, $Q = 610$, k as labeled, models as in Table 3. 95% CI: per-question normal approximation. Bootstrap: resampling the four conversations. Discordant: questions only engram / only mem0 answered correctly.

comparison	Δ (pts)	95% CI	95% CI, bootstrap	discordant	McNemar p
engram $k=3$ – mem0 $k=3$	+14.9	[+11.2, +18.7]	[+8.2, +19.8]	120 / 29	2.4e-14
engram $k=3$ – mem0 $k=6$ (token-matched)	+8.7	[+5.2, +12.1]	not computed	86 / 33	1.3e-06
engram $k=20$ – mem0 $k=20$	+0.8	[-2.3, +3.9]	[-4.5, +5.4]	49 / 44	0.679

Table 5: Held-out accuracy by LoCoMo category, 610 questions, k as labeled. Models as in Table 3.

category	Q	engram $k=3$	mem0 $k=3$	mem0 $k=6$ (matched)	engram $k=20$	mem0 $k=20$
multi-hop	110	49.1%	31.8%	40.9%	67.3%	61.8%
temporal	119	84.0%	71.4%	73.9%	88.2%	84.0%
open-domain	33	51.5%	30.3%	33.3%	51.5%	57.6%
single-hop	348	79.3%	64.9%	71.8%	82.2%	83.3%

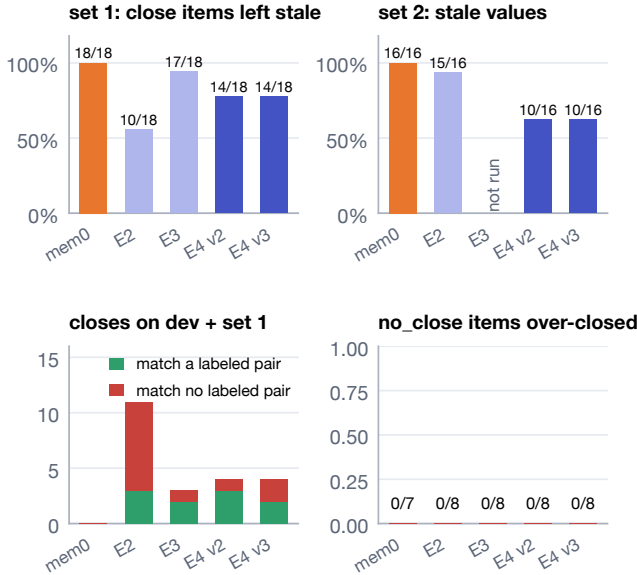


Figure 8: Store outcomes on the update sets: set-1 close items left stale, set-2 stale values, closes on dev + set 1 split by whether they match a labeled pair, and no_close items over-closed. E3 was not run on set 2.

234 times on dev + set 1 and closed 2 plans correctly and 1 wrongly.

Belief v3. On the same data, v3 ignored 36 weak answers on dev + set 1 and 39 with set 2 added. It closed the same four facts as v2 on dev + set 1. One of them, D2:5, crossed the close line at a different message (U:S01 rather than U:E11). Since that pair is not labeled, the audit scores v3 at 2 matching and 2 not matching, against v2’s 3 and 1. Stale and over-close counts are unchanged. v3 targets the weak-support failure that closed true facts under Laya (§5.6).

Hygiene. One pass cost \$0.0015 on dev + set 1 (235 decisions). On the held-out conversations it cost \$0.0112, \$0.0254, \$0.0332 and \$0.0323, making 9, 23, 32 and 24 links.

5.4 Calibration

Contradiction regression. The regression set has 50 contradiction pairs, each an old fact, a new fact and a message, labeled with accepted relations and a temporal status (bench/contradiction_pairs.jsonl). Under the current question versions:

- Jev’s relation choice is in the accepted set for 90.0% of pairs (easy 100.0%, medium 93.3%, subtle 73.3%).
- Its temporal status is right for 94.0%.
- The write path’s close rule is right for 76.0%, with 0 false closes.
- Its mean top probability is 0.88 when right and 0.81 when wrong.

Calibration error. We measure expected calibration error (ECE) on two label sets:

- 29 escalation labels, where Sonnet decided a relation Jev was unsure of
- the 50 gold pairs

On the gold pairs Jev’s relation ECE is 0.14 (fitted $T = 1.48$) and its temporal ECE is 0.04. On the escalation labels, which are by construction the cases Jev was unsure of, relation ECE is 0.15. Table 7 has the full comparison, with Laya.

Cost/error tradeoff. The 29 escalation labels are too few for a curve, so Figure 10 uses the 50 gold pairs.

- $c_J = \$0.000016$ is the mean Jev cost of one relation decision over 36,429 held-out decisions.
- $c_L = \$0.0073$ is the mean cost of one escalation over 148 logged escalations.
- No LLM run on the gold pairs is saved, so ε_L is an assumption, plotted at 0 and 0.1.

With Jev alone the error is 10.0%. At $\theta = 0.85$, 32.0% of decisions escalate, the cost is \$0.002368 per decision, and E is 6.0% ($\varepsilon_L = 0$) or 9.2% ($\varepsilon_L = 0.1$). At the production threshold of 0.60, 8.0% escalate and E is 10.0% or 10.8%.

Table 6: Store safety on dev + update set 1 and set 2 (storage metrics only). Extraction claude-haiku-4-5, decisions Jev (jev-1.13.0). Columns: set-1 no_close items over-closed / stored; set-2 keep items kept / stored; closes on dev + set 1, split into those matching a labeled pair and those not. Over-closed: a labeled no_close item whose fact was closed by its own update message. Matching a labeled pair: a close whose (closed fact’s message, closing message) is a labeled update or superseded pair. The set-2 keep set has one item.

arm	set-1 over-closed	set-2 kept	closes	labeled	unlabeled
mem0	0/7	1/1	0	0	0
E2: Jev, replace-on-update	0/8	1/1	11	3	8
E3: structural rules	0/8	not run	3	2	1
E4: belief v1	0/8	1/1	3	2	1
E4: belief v2 (frozen)	0/8	1/1	4	3	1
E4: belief v3	0/8	1/1	4	2	2

Table 7: Calibration on the escalation labels (dev + update sets, labels by claude-sonnet-4-6) and the gold contradiction pairs; Jev jev-1.13.0, Laya base checkpoint zero-shot. ECE uses 10 equal-width bins on the top choice. T is fitted by NLL per question. The last column is out of sample (2-fold). Relation probabilities fold negates into contradiction, since the pairs predate negates.

labels	question	backend	n	top-1 acc.	mean conf.	ECE	fitted T	ECE at T
escalation	relation	jev	29	79.3%	0.92	0.15	2.51	0.10
escalation	relation	laya	29	10.3%	0.47	0.37	5.41	0.12
gold pairs	relation	jev	50	82.0%	0.87	0.14	1.48	0.14
gold pairs	relation	laya	50	28.0%	0.47	0.20	4.22	0.02
gold pairs	temporal status	jev	50	94.0%	0.95	0.04	1.13	0.04
gold pairs	temporal status	laya	50	76.0%	0.72	0.06	0.74	0.07

5.5 Negative result: closing stale facts does not change answers

mem0 never closes anything and leaves every set-1 close item stale (18/18), yet answers 29/30 set-1 questions at default k. The arm that closes most aggressively leaves 10/18 stale and answers 30/30.

The answer model resolves recency itself: extraction writes dates into the memory text, and the compact rendering adds the date each fact was said. Removing all dates, validity and source from the answer context (no-dates column) still leaves every arm we ran that way at 29/30 or higher on set 1. Only the k=3 budget separates the systems (25/30 for mem0, 30/30 for engram), and §5.2 attributes that to ranking, not to closes.

Set 2, which asks about chains of changes and past moments, is at its ceiling for every arm. Current LoCoMo-style questions do not reward a correct store.

5.6 Negative result: Laya, base checkpoint, zero-shot

Everything in this section concerns the 421M base checkpoint, used zero-shot as its documentation advises against [5]. The checkpoint fine-tuned for typed decisions (reported at 0.766 on its own benchmark) and fine-tuning on our escalation labels were not tested; they are the obvious follow-up.

Regression and calibration. On the 50 regression

pairs (Table 9), Laya chooses an accepted relation for 48.0% with Jev’s question wording and 38.0% with wording rewritten to fit its 192-token question budget. Jev scores 90.0%. Laya never reaches the action threshold, so it closes nothing. Its gold-pair relation accuracy is 28.0% (Table 7). A temperature lowers its ECE but not its accuracy.

Agreement with Jev. With the frozen Jev arm replayed from cache and Laya answering every request on the side, the two gave the same answer on 44.8% of 6,764 decisions and the same action at the 0.85 threshold on 31.2%. On relation_to_candidate they agreed on 5.9% (Table 10, Figure 11).

Laya deciding every question. On dev + set 1 at k=3 it scored 24/35 LoCoMo and 26/30 update questions. Jev scored 29/35 and 30/30. It made 23 closes, of which 23 match no labeled pair, and left 43 of 66 facts active. Most of these closes came from weak duplicate and refinement answers below 0.5 lowering belief, the failure belief v3 removes (§3.4).

Hybrid. Laya answered the two high-volume yes/no questions (relevant_to_query, same_fact) and Jev the rest.

- Agreement on the routed questions: same answer 86.9%, same action 34.1%.
- Writes were identical to the all-Jev arm. At k=3 only 51.8% of Jev’s top-3 lines survived.
- It saved 42.6% of Jev’s cost on dev + set 1 (\$0.025

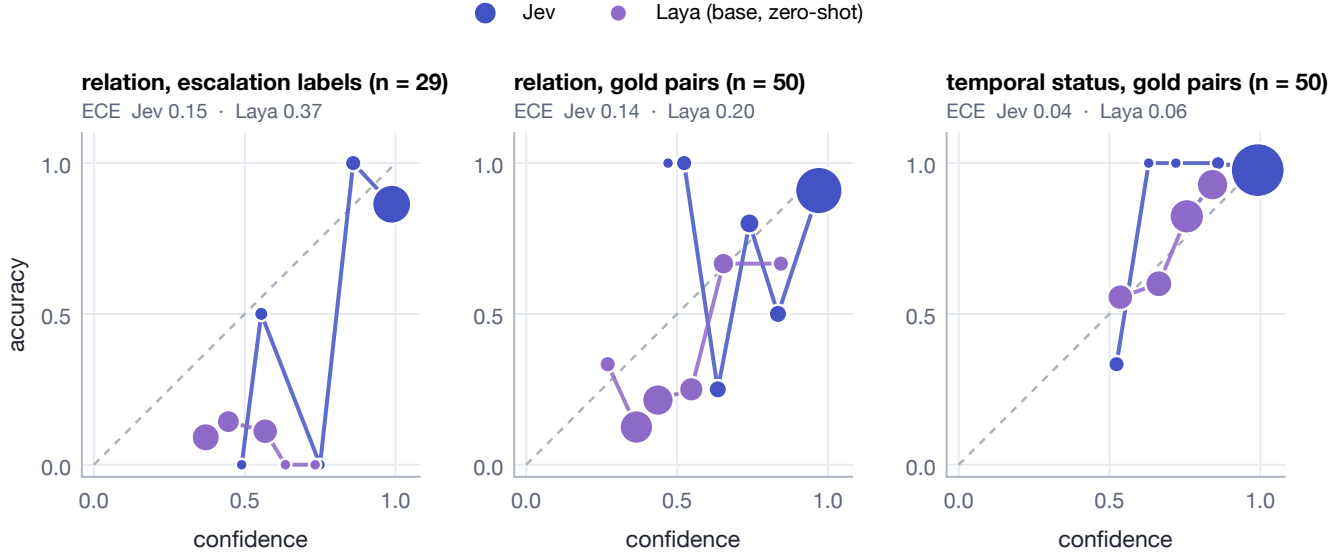


Figure 9: reliability diagrams (accuracy against confidence) for Jev and Laya on both label sets.

Table 8: Update sets on the dev slice (conv-26 sessions 1–4 plus the update messages). Set 1 has 30 update questions and set 2 has 20. Stale counts are close items whose old fact is still active, over close items stored. Accuracy is at default k (all retrieved memories) unless marked. Extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6.

arm	set-1 stale	set-2 stale	set-1 acc.	set-2 acc.	set-1 acc. k=3	set-1 acc. no dates
mem0 (add-only)	18/18	16/16	29/30	20/20	25/30	29/30
E2: Jev, replace-on-update	10/18	15/16	30/30	19/20	30/30	30/30
E4: belief v1	17/18	15/16	30/30	19/20	30/30	29/30
E4: belief v2 (frozen)	14/18	10/16	30/30	20/20	30/30	not run

Table 9: Contradiction regression (50 gold pairs), relation_to_candidate and temporal_status in one request. Laya: convaiinnovations/laya base checkpoint, zero-shot, fp16, on the local MLX server.

backend	relation exact	temporal	close rule	closes	false closes
Jev (jev-1.13.0)	90.0%	94.0%	76.0%	22	0
Laya, Jev wording (truncated)	48.0%	76.0%	32.0%	0	0
Laya, native wording (fits)	38.0%	76.0%	32.0%	0	0

Table 10: Laya against Jev on identical requests: dev + update sets 1 and 2, frozen arm’s trajectory at k=3 (write and retrieval decisions; n per row). Jev jev-1.13.0; Laya base checkpoint, zero-shot. Same action: both choose the same label at $p \geq 0.85$, or neither reaches it. Acts: share of decisions at $p \geq 0.85$.

question	n	same answer	same action	Jev acts	Laya acts
relation_to_candidate	2,779	5.9%	18.5%	79.9%	6.4%
relevant_to_query	1,440	93.5%	38.9%	89.7%	31.3%
same_fact	585	72.6%	35.2%	96.2%	34.4%
plan_fulfilled	522	66.5%	16.1%	90.0%	32.6%
worth_remembering	226	53.5%	77.0%	18.1%	4.9%
fact_kind	226	41.6%	38.1%	63.7%	15.5%
temporal_status	226	66.8%	42.5%	62.8%	5.3%
edge_type	226	41.6%	50.9%	29.2%	55.3%
durability	226	43.8%	47.3%	52.7%	0.0%
sensitivity	226	66.4%	57.5%	45.1%	7.1%
query_relation	48	54.2%	47.9%	39.6%	75.0%
relation_to_candidate_recheck	34	29.4%	41.2%	35.3%	26.5%



Figure 10: $C(\theta)$ and $E(\theta)$ on the 50 gold pairs. ε_L is assumed, not measured; the 29 escalation labels were too few for a curve.

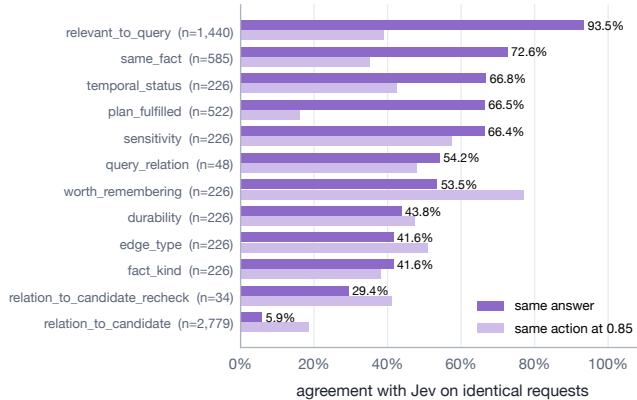


Figure 11: Agreement between the Laya base checkpoint (zero-shot) and Jev on identical requests, per question, sorted by same answer. Dev + update sets 1 and 2, frozen arm’s trajectory.

per run).

- Table 11 has accuracy.

5.7 Systems notes

Jev latency is flat in request size. Across 9,446 live Jev requests from 25 runs, median latency is between 222 ms and 269 ms for requests of 1 to 50 questions, and 371 ms for 51–80. A least-squares fit gives 380 ms plus 7.19 ms per question (Figure 12; Table 18 in Appendix D).

Variation over time is larger than variation over size. For requests of 16–20 questions, the per-run median was

Table 11: Hybrid against all-Jev: dev slice with update sets (Q per row: 35 LoCoMo, 30 set 1, 20 set 2), k as labeled, extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6.

questions	all-Jev k=3	hybrid k=3	all-Jev k=20	hybrid k=20
update set 1	30/30	27/30	30/30	30/30
LoCoMo dev	29/35	29/35	31/35	31/35
update set 2	19/20	17/20	20/20	20/20

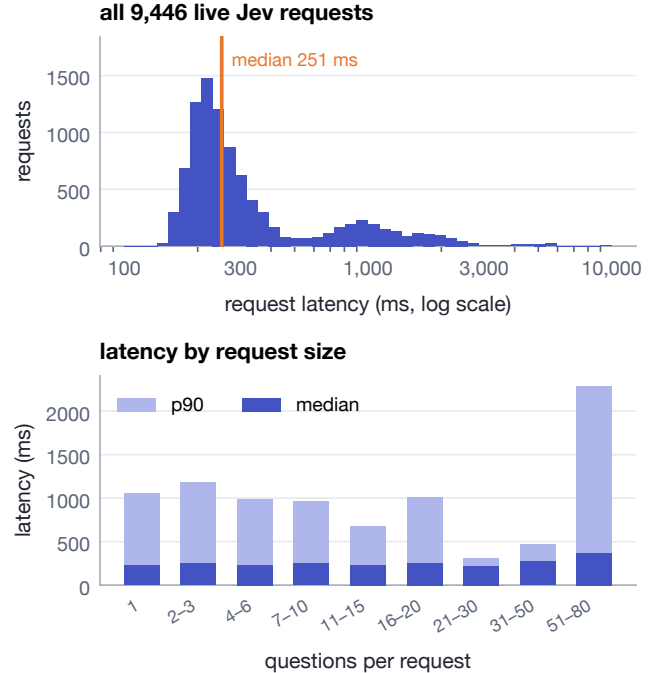


Figure 12: Jev request latency: distribution over all logged requests, and median and p90 by request size.

between 212 ms and 267 ms in 12 runs, and 793 ms and 607 ms in the two held-out runs made during one slower period.

Extraction dominates write cost. On the held-out set, engram’s write cost is \$10.22 to \$10.55 per 1,000 messages and mem0’s is \$9.92 to \$10.16. The decision layer is 3.3% to 4.8% of engram’s (Table 19, Appendix D).

6 Discussion

What the decision layer buys.

- Cost and latency (§5.1).
- An audit trail: every decision has probabilities and a version.
- Store operations that are reversible and gated (§5.3).
- A price at which re-examining the store is routine (§3.5).

What it does not buy.

- Accuracy at default k: it ties (§5.1).

- Update-question accuracy on today’s questions: every arm is near its ceiling (§5.5).

Benchmarks do not see storage correctness. A question about a fact that changed is answerable from a store that kept both versions, as long as the memories carry dates. An evaluation that rewards a correct store would need:

- questions scored against validity windows
- small retrieval budgets, where a stale fact displaces a current one
- memories stored without dates
- long chains of changes

Update set 2 targets these, and every arm is at its ceiling on it.

Rerank or larger k. Under a three-memory budget, a cheap listwise rerank is worth +8.7 points over mem0 at matched tokens. At k=20 the ranking stops mattering. For deployments that pay per context token, reranking is the cheaper route to the same accuracy.

Threats to validity.

- *One baseline and one benchmark.* We compare only mem0 OSS 2.1.0, on four held-out LoCoMo conversations.
- *Author-written update sets.* The update sets and the regression pairs were written and labeled by the system’s author.
- *One decision model.* Jev is the only decision model evaluated as the deciding backend, at one version. Laya was tested only as a zero-shot base checkpoint.
- *mem0’s date handling.* mem0’s open-source path gives no observation date, so relative dates resolve against the run date; the comparison uses it as shipped. A variant with the session date patched in scored 31/35 on dev, against 30/35 unpatched, within noise.
- *Dev-only rerank attribution.* The rerank attribution in §5.2 rests on a dev ablation.
- *One judge model.* The judge is an LLM (claude-sonnet-4-6). We did not measure its agreement with human labels.

7 Related work

Memory systems. mem0 [4] extracts and consolidates facts with LLM calls; its 2.x open-source default is add-only. Zep/Graphiti [16] builds a temporal knowledge graph and invalidates edges with an LLM. MemGPT/Letta [15] manages memory tiers through LLM function calls. A-MEM [21] links notes with LLM-written attributes, and MAGMA [9] organizes memory as multiple graphs. ByteRover [13] resolves most queries through a five-tier progressive retrieval, in under 100 ms

and without LLM calls, and escalates to agentic reasoning only for novel questions.

Typed decisions in memory. Jev-Mem [8] is the closest design: Jev controls typing, relation construction, query routing, budget allocation, traversal, candidate scoring and stopping. The two designs differ in the store and in retrieval. Jev-Mem runs with admission filtering off and preserves every observation, so its store is never updated or closed; engram’s store is governed by a close/belief policy with reversible closes and merges (§§3.3–3.4). Jev-Mem routes queries across multiple views; engram uses a single listwise rerank over cosine candidates plus a query-relation pull (§3.1). The AtMem–Jev article [18] measured the rerank at the ranking level (Recall@10 unchanged; median batch latency 3.32 s) and reported no answer accuracy or intervals.

Benchmarks. LoCoMo [11] and LongMemEval [20] evaluate long-term conversational memory. We used only LoCoMo.

Reranking and context. Long contexts are used poorly by language models [10], and LLMs rerank candidates well [17]; spending compute on selecting context rather than adding more of it follows from both, and our rerank result is an instance.

Small models as decision layers. Routers send queries to cheaper models [3, 14], and small classifiers act as guardrails [7]. The escalation rule of §3.6 is a router of that kind.

Calibration and thresholds. Temperature scaling [6] and conformal prediction [1] give principled thresholds for acting on a model’s probability. §3.4 depends on the former.

8 Conclusion

With extraction held identical, typed decisions replaced an LLM decision layer at 70.0× lower cost and 27.6× lower median decision latency, with no measured accuracy loss. On 610 held-out questions, a cheap listwise rerank gave +8.7 points over mem0 at matched context under a three-memory budget, and the systems tied at k=20. Closing stale facts, the part of the design aimed at correctness, did not change answers on current benchmarks. The base checkpoint of a small open-weights decision model, used zero-shot, did not make the relational decisions.

Release. [REPO URL PLACEHOLDER] contains:

- engram’s code (MIT license)
- mem0’s prompts as used (Apache-2.0, with attribution)
- the update sets and regression pairs
- every per-question result file in bench/results/
- the call cache needed to reproduce each table without API spend

References

- [1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [2] ByteRover. Benchmarking AI agent memory: ByteRover 2.0 scores 92.2% and rewrites the LoCoMo leaderboard. <https://byterover.dev/blog/benchmark-ai-agent-memory>, 2026. Accessed September 2026.
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. arXiv:2305.05176.
- [4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [5] Convai Innovations. Laya: a non-autoregressive system 1 decision model. Hugging Face model card, <https://huggingface.co/convaiinnovations/laya>, 2026. Accessed September 2026. ModernBERT-large, 421M parameters, 512-token context. The card states the base checkpoint is a fast base to specialise, not a zero-shot decision engine, and ships a separate typed-decisions checkpoint.
- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.
- [7] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [8] Dongming Jiang, Yi Li, and Bingzhe Li. Jev-Mem: System-one-controlled agentic memory for efficient AI agents. *arXiv preprint arXiv:2609.23986*, 2026. Concurrent work, posted 21 September 2026.
- [9] Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. MAGMA: A multi-graph based agentic memory architecture for AI agents. *arXiv preprint arXiv:2601.03236*, 2026.
- [10] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.
- [11] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.747. arXiv:2402.17753.
- [12] Mem0. LoCoMo vs. LongMemEval vs. BEAM: The 2026 AI memory benchmark guide. <https://mem0.ai/blog/ai-memory-benchmarks-in-2026>, 2026. Accessed September 2026.
- [13] Andy Nguyen, Danh Doan, Hoang Pham, Bao Ha, Dat Pham, Linh Nguyen, Hieu Nguyen, Thien Nguyen, Cuong Do, Phat Nguyen, and Toan Nguyen. ByteRover: Agent-native memory through LLM-curated hierarchical context. *arXiv preprint arXiv:2604.01599*, 2026.
- [14] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.18665.
- [15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. The system now maintained as Letta.
- [16] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [17] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of EMNLP*, 2023. arXiv:2304.09542.
- [18] Javad Taghia. Governed agent memory with structured judgment: An AtMem-Jev retrieval study. Hugging Face community article, <https://huggingface.co/blog/javadtaghia/an-atmem-and-jev-experiment-can-a-judgment-model-h>, 2026. Published 19 September 2026.
- [19] TypeSafe AI. Jev: introduction and the choice primitive. <https://docs.typesafe.ai/introduction>, 2026. Accessed September 2026.

- [20] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10813.
- [21] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

A Jev questions, all versions

Generated from `src/engram/decide/questions.py`.

durability (version 1, choice) Instructions: How long will `new_fact` likely stay true?

- **permanent**: Essentially never changes: birthplace, family ties, allergies.
- **long_term**: Stable for months or years: residence, job, diet, hobbies.
- **short_lived**: True for days or weeks: current mood, this week’s plans, a temporary situation.

edge_type (version 1, choice) Instructions: Which relation best describes how `new_fact.subject` relates to `new_fact.object`?

- **lives_in**: Current or past place of residence.
- **born_in**: Place of birth or origin.
- **works_at**: Employer or workplace.
- **has_role**: Job title, profession, or role.
- **studies_at**: School, university, or course of study.
- **member_of**: A club, team, community, or group.
- **married_to**: Spouse.
- **partner_of**: Romantic partner who is not a spouse.
- **family_of**: Parent, child, sibling, or other relative.
- **friend_of**: Friend.
- **colleague_of**: Coworker, manager, or report.
- **owns**: Possesses an object, pet, vehicle, or property.
- **uses**: Uses a tool, product, app, or service.
- **prefers**: Likes or favors something.
- **dislikes**: Dislikes or avoids something.
- **allergic_to**: Allergy or intolerance.
- **has_condition**: Health condition, injury, or medication.
- **follows_diet**: Dietary pattern (vegetarian, keto, halal, ...).

- **hobby**: A leisure activity.
- **habit**: A recurring routine.
- **goal**: Something the subject aims to achieve.
- **plans**: A scheduled or intended future action or event.
- **attended**: A past event, trip, or visit.
- **speaks**: A language.
- **related_to**: Fallback: none of the above fits.

fact_kind (version 1, choice) Instructions: What kind of personal fact is `new_fact`?

- **preference**: A like, dislike, or taste (food, music, style, tools).
- **bio**: A stable attribute of a person: where they live, age, origin, job, education, languages.
- **event**: Something that happened or will happen at a specific time.
- **relationship**: A connection between two people or between a person and an organization.
- **task**: Something the user intends or needs to do: a goal, a to-do, a plan.
- **opinion**: A belief or judgment about something, not a personal taste.

plan_fulfilled (version 1, noun) Instructions: Does the new fact report that the plan in `existing_fact` has now happened?

- **true**: `new_fact` says the planned or intended thing in `existing_fact` actually took place or was achieved.
- **false**: `new_fact` is about something else, only mentions the plan again, adds detail, or reports progress without the plan itself having happened.

query_relation (version 1, choice) Instructions: Which relation about a person is query asking about?

Options and rubrics as `edge_type` version 1, without `related_to`; plus:

- **none**: The query does not ask about one of these relations of a person (for example general knowledge).

relation_to_candidate (version 1, choice) Instructions: How does `new_fact` relate to `existing_fact`?

- **new**: They are about different things. Both can be true, and neither changes the other.
- **duplicate**: Same information, maybe worded differently. Storing `new_fact` adds nothing.
- **update**: Same attribute of the same subject, and `new_fact` gives the newer value. `existing_fact` was true before but is no longer current (for example a move, a job change, a new phone).

- **contradiction:** Both cannot be true at the same time, and `new_fact` does not describe a change over time. It directly conflicts with `existing_fact` (for example “is vegetarian” vs “favorite food is steak”).
- **refinement:** `new_fact` adds detail to `existing_fact` without making it false (for example “lives in Berlin” becoming “lives in Kreuzberg, Berlin”).

relation_to_candidate (version 2, choice) Instructions: How does `new_fact` relate to `existing_fact`?

Options and rubrics as `relation_to_candidate` version 1; plus:

- **negates:** `new_fact` says that `existing_fact` itself no longer holds: the same subject, relation and object, now stopped, ended, lost or undone (for example “User goes to a support group” vs “User doesn’t go to the support group anymore”).

relation_to_candidate_recheck (version 1, choice) Instructions: Once `new_fact` is known, what happens to `existing_fact`?

Options and rubrics as `relation_to_candidate` version 1.

relation_to_candidate_recheck (version 2, choice) Instructions: Once `new_fact` is known, what happens to `existing_fact`?

Options and rubrics as `relation_to_candidate` version 1; plus:

- **negates:** `new_fact` says that `existing_fact` itself no longer holds: the same subject, relation and object, now stopped, ended, lost or undone (for example “User goes to a support group” vs “User doesn’t go to the support group anymore”).

relevant_to_query (version 1, noul) Instructions: Does memory help answer query?

- **true:** It states or directly implies part of the answer.
- **false:** It is off-topic or only shares a keyword.

same_fact (version 1, noul) Instructions: Do `fact_a` and `fact_b` state the same information?

- **true:** Duplicate: one could be deleted without losing information.
- **false:** Distinct: each says something the other does not.

sensitivity (version 1, choice) Instructions: Which category of sensitive personal information, if any, does `new_fact` contain?

- **none:** Not sensitive.
- **health:** Physical or mental health, conditions, medication, allergies, disability.

- **financial:** Income, debt, account details, spending, salary.
- **relationship:** Romantic, sexual, or intimate family matters.
- **credentials:** Passwords, API keys, PINs, security answers, ID numbers.

temporal_status (version 1, choice) Instructions: According to `source_message`, when is `new_fact` true?

- **current:** True now. Includes a change that already happened and still holds (moved, graduated, got married, switched jobs).
- **planned:** Expected or intended to happen in the future; not true yet.
- **past:** Only about an earlier time: a finished event or former situation that says nothing about what is true now (a past trip, a former job, childhood).
- **hypothetical:** Possible, conditional, wished for, or uncertain; not stated as actually happening.

temporal_status (version 2, choice) Instructions: According to `source_message`, when is `new_fact` true?

- **current:** True now. Includes a change that already happened and still holds: a report that someone switched to, sold, replaced, stopped, quit, moved to, started or finished something describes the new state, which is current.
- **planned:** Expected or intended to happen in the future; not true yet.
- **past:** Only about an earlier time: a finished event or former situation that says nothing about what is true now (a past trip, a former job, childhood).
- **hypothetical:** Possible, conditional, wished for, or uncertain; not stated as actually happening. A report that a change happened (switched, sold, replaced, stopped, moved) is never hypothetical.

worth_remembering (version 1, noul) Instructions: Should a personal assistant store `new_fact` in long-term memory about the user?

- **true:** A stable or useful personal detail: identity, relationships, preferences, plans, commitments, health, work, recurring habits, or a notable life event.
- **false:** Small talk, greetings, filler, a transient remark with no future use, or a statement about the conversation itself.

B mem0 prompts used

Copied from the installed mem0 2.1.0 package (Apache License 2.0, © mem0.ai); pinned in `src/engram/llm/prompts_mem0.py` and checked by `tests/test_prompts_mem0.py`. The answer and judge prompts are mem0's LoCoMo evaluation prompts, in `bench/locomo_subset.py`.

```
# ROLE

You are a Memory Extractor – a precise, evidence-bound processor responsible for extracting rich, contextual memories from conversations. Your sole operation is ADD: identify every piece of memorable information and produce self-contained, contextually rich factual statements.

You extract from BOTH user and assistant messages. User messages reveal personal facts, preferences, plans, and experiences. Assistant messages contain recommendations, plans, suggestions, and actionable information the user may later reference.

Accuracy and completeness are critical. Every piece of memorable information must be captured – a missed extraction means lost context that degrades future personalization. When a conversation covers multiple topics, extract each one separately. Do not let a dominant topic cause you to miss secondary information.

# INPUTS

## New Messages

The current conversation turn(s) with "role" (user/assistant) and "content".

Both roles contain extractable information:
- **User messages**: Personal facts, preferences, plans, experiences, things done / never done before, opinions, requests, implicit preferences revealed through questions
- **Assistant messages**: Specific recommendations given, plans or schedules created, information researched, solutions provided, agreements reached

Attribute correctly: use "User" for user-stated facts. For assistant-generated content, frame in terms of the user's context (e.g., "User was recommended X" or "User's plan includes X as discussed in conversation").

Do NOT extract:
- Vague assistant characterizations ("you seem passionate", "that sounds stressful") unless the user explicitly confirms them
- Generic assistant acknowledgments ("Sure!", "Great question!")
- Assistant meta-commentary about its own capabilities

## Summary

A narrative summary of the user's profile from prior conversations. May be empty for new users. Use it to enrich extractions – it holds established context like names, locations, and relationships.

## Recently Extracted Memories

Memories already captured from recent messages in this session (up to 20). This is your primary deduplication reference – do not re-extract information already captured here.

## Existing Memories

Memories currently in the system relevant to this conversation. Formatted as:
[{"id": "uuid-string", "text": "..."}, ...]
```

The remaining lines are in `src/engram/llm/prompts_mem0.py` (ADDITIVE_EXTRACTION_PROMPT), verbatim from mem0 2.1.0.

```
You are a smart memory manager which controls the memory of a system.
You can perform four operations: (1) add into the memory, (2) update the memory, (3) delete from the memory, and (4) no change.

Based on the above four operations, the memory will change.

Compare newly retrieved facts with the existing memory. For each new fact, decide whether to:
- ADD: Add it to the memory as a new element
- UPDATE: Update an existing memory element
- DELETE: Delete an existing memory element
- NONE: Make no change (if the fact is already present or irrelevant)

There are specific guidelines to select which operation to perform:

1. **Add**: If the retrieved facts contain new information not present in the memory, then you have to add it by generating a new ID in the id field.
- **Example**:
  - Old Memory:
    [
      {
        "id" : "0",
        "text" : "User is a software engineer"
      }
    ]
```

```

- Retrieved facts: ["Name is John"]
- New Memory:
  {
    "memory" : [
      {
        "id" : "0",
        "text" : "User is a software engineer",
        "event" : "NONE"
      },
      {
        "id" : "1",
        "text" : "Name is John",
        "event" : "ADD"
      }
    ]
  }

```

2. ****Update****: If the retrieved facts contain information that is already present in the memory but the information is totally different, then you have to update it.

If the retrieved fact contains information that conveys the same thing as the elements present in the memory, then you have to keep the fact which has the most information.

Example (a) -- if the memory contains "User likes to play cricket" and the retrieved fact is "Loves to play cricket with friends", then update the memory with the retrieved facts.

Example (b) -- if the memory contains "Likes cheese pizza" and the retrieved fact is "Loves cheese pizza", then you do not need to update it because they convey the same information.

If the direction is to update the memory, then you have to update it.

Please keep in mind while updating you have to keep the same ID.

Please note to return the IDs in the output from the input IDs only and do not generate any new ID.

- ****Example****:

```

- Old Memory:
  [
    {
      "id" : "0",
      "text" : "I really like cheese pizza"
    },
    {
      "id" : "1",
      "text" : "User is a software engineer"
    },
    {
      "id" : "2",
      "text" : "User likes to play cricket"
    }
  ]
- Retrieved facts: ["Loves chicken pizza", "Loves to play cricket with friends"]
- New Memory:
  {
    "memory" : [
      {
        "id" : "0",
        "text" : "Loves cheese and chicken pizza",
        "event" : "UPDATE",
        "old_memory" : "I really like cheese pizza"
      },
      {
        "id" : "1",
        "text" : "User is a software engineer",
        "event" : "NONE"
      },
      {
        "id" : "2",
        "text" : "Loves to play cricket with friends",
        "event" : "UPDATE",
        "old_memory" : "User likes to play cricket"
      }
    ]
  }

```

3. ****Delete****: If the retrieved facts contain information that contradicts the information present in the memory, then you have to delete it. Or if the direction is to delete the memory, then you have to delete it.

Please note to return the IDs in the output from the input IDs only and do not generate any new ID.

- ****Example****:

```

- Old Memory:
  [
    {
      "id" : "0",
      "text" : "Name is John"
    },
    {
      "id" : "1",
      "text" : "Loves cheese pizza"
    }
  ]

```

```

- Retrieved facts: ["Dislikes cheese pizza"]
- New Memory:
  {
    "memory" : [
      {
        "id" : "0",
        "text" : "Name is John",
        "event" : "NONE"
      },
      {
        "id" : "1",
        "text" : "Loves cheese pizza",
        "event" : "DELETE"
      }
    ]
  }

```

4. ****No Change****: If the retrieved facts contain information that is already present in the memory, then you do not need to make any changes.

- ****Example****:

```

- Old Memory:
  [
    {
      "id" : "0",
      "text" : "Name is John"
    },
    {
      "id" : "1",
      "text" : "Loves cheese pizza"
    }
  ]
- Retrieved facts: ["Name is John"]
- New Memory:
  {
    "memory" : [
      {
        "id" : "0",
        "text" : "Name is John",
        "event" : "NONE"
      },
      {
        "id" : "1",
        "text" : "Loves cheese pizza",
        "event" : "NONE"
      }
    ]
  }

```

You are an intelligent memory assistant tasked with retrieving accurate information from conversation memories.

CONTEXT:

You have access to memories from two speakers in a conversation. These memories contain timestamped information that may be relevant to answering the question.

INSTRUCTIONS:

1. Carefully analyze all provided memories from both speakers
2. Pay special attention to the timestamps to determine the answer
3. If the question asks about a specific event or fact, look for direct evidence in the memories
4. If the memories contain contradictory information, prioritize the most recent memory
5. If there is a question about time references (like "last year", "two months ago", etc.), calculate the actual date based on the memory timestamp. For example, if a memory from 4 May 2022 mentions "went to India last year," then the trip occurred in 2021.
6. Always convert relative time references to specific dates, months, or years. For example, convert "last year" to "2022" or "two months ago" to "March 2023" based on the memory timestamp. Ignore the reference while answering the question.
7. Focus only on the content of the memories from both speakers. Do not confuse character names mentioned in memories with the actual users who created those memories.
8. The answer should be less than 5-6 words.

APPROACH (Think step by step):

1. First, examine all memories that contain information related to the question
2. Examine the timestamps and content of these memories carefully
3. Look for explicit mentions of dates, times, locations, or events that answer the question
4. If the answer requires calculation (e.g., converting relative time references), show your work
5. Formulate a precise, concise answer based solely on the evidence in the memories
6. Double-check that your answer directly addresses the question asked
7. Ensure your final answer is specific and avoids vague time references

Memories for {speakers}:

{memories}

Question: {question}

Answer :

Your task is to label an answer to a question as 'CORRECT' or 'WRONG'. You will be given the following data:

- (1) a question (posed by one user to another user),
- (2) a 'gold' (ground truth) answer,
- (3) a generated answer

which you will score as CORRECT/WRONG.

The point of the question is to ask about something one user should know about the other user based on their prior conversations.

The gold answer will usually be a concise and short answer that includes the referenced topic, for example:

Question: Do you remember what I got the last time I went to Hawaii?

Gold answer: A shell necklace

The generated answer might be much longer, but you should be generous with your grading - as long as it touches on the same topic as the gold answer, it should be counted as CORRECT.

For time related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references (like "last Tuesday" or "next month"), but you should be generous with your grading - as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs (e.g., "May 7th" vs "7 May"), consider it CORRECT if it's the same date.

Now it's time for the real question:

Question: {question}

Gold answer: {gold_answer}

Generated answer: {generated_answer}

First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG.

Do NOT include both CORRECT and WRONG in your response, or it will break the evaluation script.

Just return the label CORRECT or WRONG in a json format with the key as "label".

C Update sets

Set 1 (bench/updates_conv26.json, 30 items). Columns: id, tier, label, the original fact (and its message), the update message, the question and the gold answer.

id	tier	label	original	update	question	gold
E01	easy	close	Melanie's main creative outlet is painting, which relaxes her after a long day. (D1:16)	I haven't painted in weeks, honestly. Pottery is my main creative outlet now - I go to the studio three evenings a week.	What is Melanie's main creative outlet now?	Pottery (she has stopped painting)
E02	easy	close	Melanie's favorite painting is the lake sunrise she painted, which is special to her. (D1:14)	Funny thing - the lake sunrise isn't my favorite anymore. The forest-at-dusk piece I finished last week is my favorite now.	Which of her own paintings does Melanie consider her favorite now?	The forest-at-dusk painting
E03	easy	close	Melanie has a job and is swamped with work and the kids. (D1:2)	Big change: I quit my job last Friday so I can be home with the kids full-time.	Does Melanie still have a job?	No, she quit to stay home with the kids
E04	easy	close	Caroline has the necklace her grandma gave her. (D4:3)	I'm heartbroken - I lost my grandma's necklace at the beach on Saturday. We searched for hours but it's gone.	Does Caroline still have her grandmother's necklace?	No, she lost it at the beach
E05	easy	close	Caroline has a hand-painted bowl a friend made for her 18th birthday. (D4:5)	My hand-painted bowl from my 18th birthday fell off the shelf and shattered. I kept a shard but the bowl is gone.	Does Caroline still have the hand-painted bowl from her 18th birthday?	No, it broke
E06	easy	close	Caroline goes to an LGBTQ support group that makes her feel accepted. (D1:7)	I don't go to the LGBTQ support group anymore - it disbanded after the organizer moved away.	Does Caroline still go to the LGBTQ support group?	No, the group disbanded
E07	easy	close	Melanie goes swimming with her kids. (D1:18)	The pool near us closed for good, so the kids and I don't swim anymore. We do bike rides together instead.	Does Melanie still go swimming with her kids?	No, the pool closed; they go on bike rides instead

id	tier	label	original	update	question	gold
E08	easy	close	Melanie's two younger kids love nature. (D4:8)	The two younger ones have moved on from nature stuff - now they're completely obsessed with dinosaurs.	What are Melanie's two younger kids into now?	Dinosaurs
E09	easy	close	Melanie plays the violin during her me-time. (D2:5)	I sold my violin last month. I'm learning guitar now instead - it's more fun to play with the kids.	What instrument does Melanie play now?	Guitar (she sold her violin)
E10	easy	close	Melanie runs during her me-time. (D2:5)	My knee has been bad, so I stopped running completely. I swim laps at the gym now.	What exercise does Melanie do now instead of running?	Swimming laps
E11	easy	close	Melanie reads during her me-time. (D2:5)	I've swapped reading for meditation in my me-time - twenty minutes every morning.	What does Melanie do in her me-time now instead of reading?	Meditation
E12	easy	close	Caroline chose an adoption agency because it supports LGBTQ+ people. (D2:12)	Update on adoption: I switched agencies. I'm with Rainbow Families now - they're closer to home and move faster.	Which adoption agency is Caroline working with now?	Rainbow Families
E13	easy	close	Caroline's plan is to adopt kids and give them a loving home. (D2:10)	I've changed my plan a bit - I'm going to foster first instead of adopting right away, to learn the ropes.	Is Caroline adopting right away or fostering first?	Fostering first
E14	easy	close	Melanie's self-care is still a work in progress. (D2:7)	Remember how self-care was a work in progress for me? Not anymore - I finally have a routine that sticks.	Is self-care still a work in progress for Melanie?	No, she now has a routine that sticks
E15	easy	close	Caroline's grandma lives in Sweden. (D4:3)	My grandma moved out of Sweden - she lives in Norway with my aunt now.	Where does Caroline's grandma live now?	Norway
S01	subtle	no__ close	Melanie has been married to her husband for 5 years. (D3:16)	Our first apartment after the wedding was so tiny - we still laugh about it.	Is Melanie still married?	Yes, married for 5 years
S02	subtle	no__ close	Caroline is pursuing a career in counseling and mental health. (D4:11)	Back in high school I actually wanted to be a nurse. Funny how things turn out.	What career is Caroline pursuing?	Counseling / mental health, working with trans people
S03	subtle	no__ close	Caroline has known her current friends for 4 years, since she moved from her home country. (D3:13)	My first year here I was pretty lonely, before I met my friends.	How long has Caroline had her current group of friends?	4 years
S04	subtle	no__ close	Caroline moved away from Sweden, her home country. (D3:13)	Someday I might move back to Sweden, but not anytime soon.	Has Caroline moved back to Sweden?	No, it's only a someday idea
S05	subtle	no__ close	Melanie ran a charity race for mental health. (D2:1)	I want to run another charity race in the fall - maybe a half marathon this time.	Has Melanie run a second charity race yet?	No, she plans one for the fall
S06	subtle	no__ close	Melanie took her family camping in the mountains. (D4:6)	Next summer we hope to camp at the Grand Canyon, if we can save up.	Where did Melanie most recently go camping with her family?	The mountains
S07	subtle	no__ close	Caroline attended an LGBTQ+ counseling workshop. (D4:13)	That counseling workshop was the first one I'd ever been to.	Has Caroline attended an LGBTQ+ counseling workshop?	Yes
S08	subtle	no__ close	Caroline is an openly transgender woman who shares her journey. (D1:5)	Before I came out, I was too scared to tell anyone about myself.	Is Caroline open about being transgender?	Yes
S09	subtle	no__ close	Caroline plans to adopt as a single parent. (D2:14)	I used to think I'd only adopt with a partner someday.	Is Caroline planning to raise kids as a single parent?	Yes
S10	subtle	no__ close	Melanie wants to be courageous for her family. (D3:10)	I used to be really shy, even as a teenager.	Does Melanie want to be courageous for her family?	Yes
F01	fulfilled	close__ fulfilled	Caroline plans to continue her education. (D1:9)	I did it - I enrolled in a psychology certificate program at the community college. Classes start next week!	Has Caroline enrolled in a program to continue her education?	Yes, a psychology certificate program at the community college
F02	fulfilled	close__ fulfilled	Melanie's kids are looking forward to summer break. (D2:7)	The kids' summer break finally started and they are loving every minute of it.	Has Melanie's kids' summer break started?	Yes

id	tier	label	original	update	question	gold
F03	fulfilled	close__fulfilled	Caroline is thinking of working with trans people. (D4:13)	I started volunteering as a peer counselor for trans youth at the community center this week.	Has Caroline started working with trans people?	Yes, she volunteers as a peer counselor for trans youth
F04	fulfilled	close__fulfilled	Caroline is researching adoption agencies. (D2:8)	I finished my research and submitted my application to Rainbow Families on Monday.	Has Caroline submitted an adoption application?	Yes, to Rainbow Families
F05	fulfilled	close__fulfilled	Caroline plans to keep using her voice to share her story. (D3:7)	I gave another talk about my journey, this time at a high school in the next town.	Has Caroline given another talk about her journey?	Yes, at a high school in the next town

Set 2 (bench/updates2_conv26.json). Messages and questions as stored:

message	speaker	date	text
U2:C01.1	Caroline	11:00 am on 3 September, 2023	I just moved into a studio apartment downtown.
U2:C01.2	Caroline	11:00 am on 1 October, 2023	Moved again! I'm in a two-bedroom in Oak Park now, so there's room for a foster kid.
U2:C01.3	Caroline	11:00 am on 12 November, 2023	We finally found a house - I live in Evanston now.
U2:C02.1	Melanie	11:00 am on 5 September, 2023	My guitar teacher is Ben, he's really patient with me.
U2:C02.2	Melanie	11:00 am on 28 September, 2023	I switched guitar teachers - Priya teaches me now.
U2:C02.3	Melanie	11:00 am on 20 October, 2023	Priya moved away, so I'm learning guitar from YouTube videos for now.
U2:C02.4	Melanie	11:00 am on 20 November, 2023	I joined a group guitar class at the community center instead of YouTube.
U2:C03.1	Caroline	11:00 am on 10 September, 2023	I passed my foster home study!
U2:C03.2	Caroline	11:00 am on 8 October, 2023	I've been matched with a 7-year-old named Leo.
U2:C03.3	Caroline	11:00 am on 5 November, 2023	Leo went back to live with his grandmother, which is the best thing for him.
U2:C03.4	Caroline	11:00 am on 3 December, 2023	Big news: I've been matched with two siblings, Maya and Sam.
U2:C04.1	Melanie	11:00 am on 7 September, 2023	My oldest started at Lincoln Middle School this week.
U2:C04.2	Melanie	11:00 am on 15 October, 2023	We moved our oldest to Riverside Academy - Lincoln wasn't a good fit.
U2:C04.3	Melanie	11:00 am on 25 November, 2023	Our oldest is back at Lincoln Middle, Riverside was too far.
U2:C05.1	Caroline	11:00 am on 12 September, 2023	I volunteer at the community center on Tuesdays.
U2:C05.2	Caroline	11:00 am on 3 October, 2023	I switched my volunteer shift to Thursdays.
U2:C05.3	Caroline	11:00 am on 1 November, 2023	My volunteer shift moved to Saturday mornings.
U2:C05.4	Caroline	11:00 am on 6 December, 2023	I cut back my volunteering to every other Saturday morning.
U2:N01.1	Caroline	11:00 am on 4 September, 2023	My favorite coffee spot is Blue Door Café.
U2:N01.2	Caroline	11:00 am on 30 October, 2023	My favorite coffee spot is Grind House.
U2:N02.1	Melanie	11:00 am on 6 November, 2023	We drive a blue minivan.
U2:N02.2	Melanie	11:00 am on 2 November, 2023	We drive a silver SUV.
U2:N03.1	Caroline	11:00 am on 9 September, 2023	My therapist is Dr. Alvarez.
U2:N03.2	Caroline	11:00 am on 6 November, 2023	My therapist is Dr. Kim.
U2:N04.1	Melanie	11:00 am on 11 September, 2023	Our book club meets at Anna's house.
U2:N04.2	Melanie	11:00 am on 9 November, 2023	Our book club meets at the public library.
U2:N05.1	Caroline	11:00 am on 14 September, 2023	My phone is an iPhone 12.
U2:N05.2	Caroline	11:00 am on 14 November, 2023	My phone is a Pixel 8.

id	type	question	gold	evidence
P01	point_in_time	What instrument did Melanie play in June 2023?	Violin	E09: violin until 19 July 2023, then guitar
P02	point_in_time	Where did Caroline's grandma live in May 2023?	Sweden	E15: Sweden until 31 July 2023, then Norway
P03	point_in_time	Which adoption agency was Caroline working with in early July 2023?	The agency she chose for its LGBTQ+ inclusivity (before switching to Rainbow Families)	E12: switched 25 July 2023
P04	point_in_time	Did Melanie have a job in June 2023?	Yes	E03: quit around 6 July 2023
P05	point_in_time	What was Melanie's main creative outlet in May 2023?	Painting	E01: pottery from 2 July 2023
C01q1	chain_current	Where does Caroline live now?	A house in Evanston	
C01q2	chain_point_in_time	Where was Caroline living in October 2023?	A two-bedroom in Oak Park	
C02q1	chain_current	How is Melanie learning guitar now?	A group class at the community center	
C02q2	chain_point_in_time	Who was teaching Melanie guitar in early October 2023?	Priya	
C03q1	chain_current	Which children is Caroline currently matched with?	Maya and Sam (siblings)	

id	type	question	gold	evidence
C03q2	chain_ point_in_ time	Who was Caroline matched with in October 2023?	Leo, a 7-year-old	
C04q1	chain_ current	Which school does Melanie's oldest child attend now?	Lincoln Middle School	
C04q2	chain_ point_in_ time	Which school was Melanie's oldest at in late October 2023?	Riverside Academy	
C05q1	chain_ current	When does Caroline volunteer at the community center now?	Every other Saturday morning	
C05q2	chain_ point_in_ time	What day did Caroline volunteer in mid-October 2023?	Thursdays	
N01	no_ temporal_ cue	What is Caroline's favorite coffee spot?	Grind House	
N02	no_ temporal_ cue	What car does Melanie's family drive?	A silver SUV	
N03	no_ temporal_ cue	Who is Caroline's therapist?	Dr. Kim	
N04	no_ temporal_ cue	Where does Melanie's book club meet?	The public library	
N05	no_ temporal_ cue	What phone does Caroline have?	A Pixel 8	

D Per-conversation held-out tables

Table 15: D.1 Held-out, per conversation, k=3. Q per row; models as in Table 3.

conv.	Q	engram	mem0	Δ (questions)	engram tokens / q	mem0 tokens / q
conv-30	81	59/81	55/81	4	285	154
conv-41	152	113/152	89/152	24	297	161
conv-42	199	150/199	111/199	39	292	163
conv-43	178	125/178	101/178	24	283	153

Table 16: D.2 Held-out, per conversation: engram k=3 against token-matched mem0 (k=6). Q per row; models as in Table 3.

conv.	Q	engram k=3	mem0 k=6	Δ (questions)
conv-30	81	59/81	54/81	5
conv-41	152	113/152	109/152	4
conv-42	199	150/199	119/199	31
conv-43	178	125/178	112/178	13

Table 17: D.3 Held-out, per conversation, k=20. Q per row; models as in Table 3.

conv.	Q	engram	mem0	Δ (questions)	engram tokens / q	mem0 tokens / q
conv-30	81	62/81	63/81	-1	1,434	996
conv-41	152	128/152	135/152	-7	1,521	1,049
conv-42	199	156/199	146/199	10	1,491	1,043
conv-43	178	136/178	133/178	3	1,389	995

Table 18: Jev (jev-1.13.0) latency by request size over all logged runs (dev, stress and held-out slices), from the decision logs (bench/results/jev_latency.json). Client-measured, after the rate limiter, retries included.

questions / request	requests	median input tokens	median latency	p90 latency
1	1,305	564	229 ms	1,052 ms
2–3	1,054	870	248 ms	1,176 ms
4–6	775	1,562	231 ms	992 ms
7–10	1,417	4,314	246 ms	967 ms
11–15	227	2,432	231 ms	680 ms
16–20	3,240	5,593	255 ms	1,009 ms
21–30	184	3,551	222 ms	312 ms
31–50	505	6,903	269 ms	472 ms
51–80	739	9,187	371 ms	2,290 ms

Table 19: Held-out write side (conv-30, 41, 42, 43; writes only). Extraction claude-haiku-4-5 for both systems; decisions Jev jev-1.13.0 with claude-sonnet-4-6 escalations. One ingestion per system per conversation. Decision layer = Jev plus escalations.

conv.	engram \$/1k	decision \$/1k	mem0 \$/1k	decision p50	engram write p50	mem0 write p50	engram facts	mem0 memories
conv-30	\$10.22	\$0.33	\$9.92	1,982 ms	928 ms	1,034 ms	267	276
conv-41	\$10.54	\$0.47	\$10.16	1,614 ms	1,775 ms	1,437 ms	668	822
conv-42	\$10.55	\$0.50	\$10.04	981 ms	1,824 ms	1,300 ms	708	695
conv-43	\$10.45	\$0.50	\$9.96	868 ms	1,819 ms	1,210 ms	628	636

E Reproduction

All commands run from the repository root. With the call cache (bench/.cache/calls.sqlite) in place, every command below replays at no API cost.

table / figure	command
Table 2; Fig. 4	python -m bench.run --arm {e2_jev,e2_llm,mem0} --slice dev
Tables 3–5, D; Figs. 5–7	python -m bench.run --arm {e4_belief_v2,mem0} --slice heldout:<conv> --top-k 3 --also-top-k 20, then python -m bench.heldout_report and python -m bench.token_match
Tables 6, 8; Figs. 3, 8	python -m bench.run --arm <arm> --slice dev_updates[2] [--top-k 3] [--no-dates]
Tables 7, 9; Figs. 9, 11	python bench/test_contradictions.py --backend laya [--native] --save . . . , python -m bench.calibration, python -m bench.jev_regression (the Laya rows need bench/laya_server.py running)
Fig. 10	python -m bench.tradeoff
Tables 10–11	python -m bench.laya_report, python -m bench.hybrid_report
Table 18, Fig. 12	python -m bench.jev_latency
this paper	python paper/build.py, python paper/figures.py

The spend ledger records cost per run, not per table. Per-arm totals:

arm	runs	Jev	Claude
mem0	15	\$0.0000	\$35.94
e4_belief_v2	18	\$1.0917	\$31.80
e2_jev	2	\$0.0636	\$3.51
e2_llm	2	\$0.0134	\$3.21
mem0_token_matched	1	\$0.0000	\$2.52
e4_belief_v2_laya	2	\$0.0405	\$2.40
e2_jev_v2	4	\$0.0362	\$2.35
e1_recall	4	\$0.0653	\$2.05
e4_belief	6	\$0.0316	\$1.69
e3_structural_v2	1	\$0.0278	\$1.44
e0_baseline	8	\$0.0736	\$1.36
mem0_dated	1	\$0.0000	\$0.96
e3_structural_v3	2	\$0.0272	\$0.76
e4_belief_v2_hybrid	4	\$0.0000	\$0.73
e4_belief_v2_compact	2	\$0.0000	\$0.59
e4_belief_v2_norerank	1	\$0.0000	\$0.25
e4_belief_v2_shadow	2	\$0.0000	\$0.18
e3_structural	1	\$0.0008	\$0.04
e4_belief_v2_nohist	1	\$0.0000	\$0.01
e4_belief_v3	3	\$0.0004	\$0.00